FIGBLOOM FLEET

# Migrating from MongoDB to PostgreSQL

A step-by-step guide for this codebase: what changes, in what order, and how to switch over safely without losing data.

| Item | Today |
|---|---|
| Backend | Node.js + Express, Mongoose 8 (backend/), deployed on Render |
| Database | MongoDB Atlas: 20 collections, about 17,800 documents, ~4 MB of data |
| Data access | About 300 Mongoose calls in 27 files, incl. 10 aggregation pipelines and 35 populate() joins |
| Clients | Next.js dashboard (frontend/) and the Expo driver app; both read **_id** fields from the API |
| Target | PostgreSQL 16 + Prisma ORM, same REST API, same JSON responses |

> **Golden rule:** the dashboard and the driver app must not notice the switch. Every endpoint keeps the same URL and returns the same JSON shape (including **_id**). Only the code behind the API changes.

**Estimated effort:** 6 to 10 working days for one developer, plus 1 day for testing and switch-over. The data copy itself takes seconds; the work is rewriting the queries.

# Contents


1. Before you start: backups and decisions
2. Set up PostgreSQL and Prisma
3. Design the tables (all 20 collections)
4. Write the Prisma schema
5. Keep the API shape: the _id adapter
6. Rewrite the queries (Mongoose to Prisma cheat sheet)
7. Replace MongoDB-only features
8. Order of work: file by file
9. Copy the data across
10. Test everything
11. Switch over on Render (and roll back if needed)
12. Checklist

# 1. Before you start: backups and decisions

## Back up MongoDB

Take a full backup before touching anything, and keep it until PostgreSQL has run in production for a few weeks. Use the MongoDB Database Tools (**mongodump**) with the connection string from **backend/.env**:

```
mongodump --uri="$MONGODB_URI" --out=backup-2026-10-27
# restore later, if ever needed:
mongorestore --uri="$MONGODB_URI" backup-2026-10-27
```

## Work on a branch, follow the team workflow

Create **feature/postgres-migration** from an up-to-date **main**. Merge it into **dev-francis**, then **dev-staging** and **main** only after testing (Mike approves the last two). Keep MongoDB running in production the whole time.

```
git checkout main && git pull
git checkout -b feature/postgres-migration
```

## Decide where PostgreSQL lives

| Option | Free tier | Notes |
|---|---|---|
| Neon | Permanent free tier (0.5 GB) | Serverless Postgres, branches per environment, good fit for Render. **Recommended.** |
| Supabase | Permanent free tier (500 MB) | Postgres plus extras you don't need here; pauses after a week idle on free. |
| Render Postgres | Free instance expires after 30 days | Same dashboard as the backend; pay for a permanent instance. |

At ~4 MB of data today, every free tier has years of headroom. The **locationpings** table grows fastest (16,783 rows now); the existing nightly retention job keeps it bounded.

## Decisions already made in this guide

- **Keep the existing IDs.** Every MongoDB ObjectId (24 hex characters, e.g. `6ad9c227653d9d63d4d1e92e`) becomes the row's primary key as text. Links in the apps, tracking tokens and foreign keys all keep working.
- **Prisma ORM** for type-safe queries and migrations. Raw SQL only for the few reports that need it.
- **Small nested objects become JSONB columns** (settings blobs); **anything you filter on becomes a real column**.
- **Same JSON out of the API**, via a small adapter that renames `id` to `_id`.

# 2. Set up PostgreSQL and Prisma

1. Create the database (e.g. on Neon) and copy its connection string.
2. Add it to **backend/.env** as `DATABASE_URL` (keep `MONGODB_URI` for now; you need it to copy data).
3. Install Prisma in the backend and create the schema file:

```
cd backend
npm install @prisma/client
npm install --save-dev prisma
npx prisma init --datasource-provider postgresql
```

That creates **backend/prisma/schema.prisma**. Add one shared client so every file uses the same connection pool:

```
// backend/lib/prisma.js
import { PrismaClient } from "@prisma/client";

export const prisma = new PrismaClient();
```

Then, in **Index.js**, replace `connectDB()` (mongoose.connect) with `await prisma.$connect()` once the switch is done. Until then both can run side by side.

> **Docker:** the local stack runs the backend in **fleet-backend-container**. Add `npx prisma generate` to the backend image's build (or the container start command) so the generated client exists inside the container.

# 3. Design the tables (all 20 collections)

How each MongoDB collection maps to PostgreSQL. The field list comes from the actual Mongoose models in **backend/models/**.

| Collection (docs) | PostgreSQL table | Links to | How to handle it |
|---|---|---|---|
| **Company** | companies | Geofence, Upload | alertSettings, trackingHours, dailyStartPoint → JSONB (read/written whole). Unique slug. |
| **User** (14) | users | Company, Upload | trackingConsent → two columns (consent_accepted_at, consent_version). Unique email. |
| **Vehicle** (7) | vehicles | Company, Geofence, User | device.* → real columns (device_last_seen_at is filtered by the offline sweep). Unique (company_id, plate_number); unique device_id. |
| **LocationPing** (16,783) | location_pings | Vehicle | Largest table. Index (vehicle_id, recorded_at DESC). Consider BIGINT identity id for new rows later. |
| **Trip** (98) | trips | User, Vehicle | Plain columns. Index (vehicle_id, start_time). |
| **Alert** (79) | alerts | Company, Vehicle | TTL index on resolvedAt → scheduled DELETE job (section 7). |
| **DrivingEvent** (85) | driving_events | Company, User, Vehicle | Plain columns; feeds driver scores. |
| **Job** (1) | jobs | Company, User, Vehicle | proof (subdocument) → JSONB, or a job_proofs table if you need to search it. photoUploadIds → TEXT[]. Unique tracking_token. |
| **Inspection** (1) | inspections + inspection_items | Company, User, Vehicle | items[] → child table (one row per checklist item, with photo_upload_id). |
| **Geofence** (2) | geofences | Company | polygon[] → JSONB array of {lat,lng}. (Optional later: PostGIS.) |
| **Route** (2) | routes | Company, Vehicle | waypoints[] → JSONB; daysOfWeek → INT[]. |
| **RouteCompletion** (1) | route_completions | Company, Route, User, Vehicle | Plain columns. |
| **Expense** (5) | expenses | Company, Upload, User, Vehicle | Plain columns; amount as NUMERIC(12,2). |
| **MaintenanceRecord** (0) | maintenance_records | Company, Vehicle | Plain columns (0 documents today). |
| **Message** (8) | messages | Company, User | Plain columns. |
| **Upload** (6) | uploads | Company, User | data (Buffer) → BYTEA. Images stay in the database. |

| Collection (docs) | PostgreSQL table | Links to | How to handle it |
|---|---|---|---|
| **RefreshToken** (233) | refresh_tokens | User | Unique token_hash; index expires_at (cleanup job). |
| **Transaction** (4) | transactions | Company | Unique reference (Paystack); amount NUMERIC. |
| **PlatformSettings** | platform_settings | — | Single row; vehicleLimits → JSONB. |
| **AgentScanLog** (543) | agent_scan_logs | — | details[] → JSONB. |

### Type mapping rules

| Mongoose | PostgreSQL / Prisma | Notes |
|---|---|---|
| ObjectId (_id) | TEXT primary key (String @id) | Keep existing 24-char ids; new rows get a cuid() id (see section 4). |
| ObjectId ref | TEXT foreign key + relation | ON DELETE rules: decide per relation (most: SET NULL or RESTRICT). |
| String with enum | Prisma enum, or TEXT + CHECK | Alert.type has many values: TEXT + validation in code is simpler. |
| Number | DOUBLE PRECISION, INT, or NUMERIC | Money (expenses, transactions, revenue) → NUMERIC(12,2). |
| Date | TIMESTAMPTZ (DateTime) | Always timestamptz. Nairobi-time logic stays in code as today. |
| Boolean | BOOLEAN | |
| Buffer | BYTEA (Bytes) | Upload.data. |
| [String] / [Number] | TEXT[] / INT[] | e.g. disabledTypes, daysOfWeek. |
| Small nested object | JSONB (Json) | When it's always read/written as a whole. |
| Array of subdocuments | Child table or JSONB | Child table if you query inside it; JSONB otherwise. |
| timestamps: true | createdAt @default(now()), updatedAt @updatedAt | |

## 4. Write the Prisma schema

Write **prisma/schema.prisma** model by model, starting with the ones everything else links to. Example for the core four (abridged). **@default(cuid())** only applies to new rows: the data copy (section 9) inserts every existing record with its original 24-character MongoDB id.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Company {
  id               String    @id @default(cuid())
  name             String
  slug             String    @unique
  status           String    @default("pending")
  subscriptionPlan String    @default("trial")
  industry         String?
  alertSettings    Json?
  trackingHours    Json?
  dailyStartPoint  Json?
  logoUploadId     String?
  createdAt        DateTime  @default(now())
  updatedAt        DateTime  @updatedAt
  users            User[]
  vehicles         Vehicle[]
  @@map("companies")
}

model User {
  id                 String    @id @default(cuid())
  email              String    @unique
  passwordHash       String
  fullName           String
  role               String    // PLATFORM_ADMIN | COMPANY_OWNER | ADMIN | DRIVER
  adminAccess        String    @default("full")
  phone              String?
  avatarUploadId     String?
  consentAcceptedAt  DateTime?
  consentVersion     String?
  companyId          String?
  company            Company?  @relation(fields: [companyId], references: [id])
  createdAt          DateTime  @default(now())
  updatedAt          DateTime  @updatedAt
  @@index([companyId])
  @@map("users")
}

model Vehicle {
  id                 String    @id @default(cuid())
  companyId          String
  company            Company   @relation(fields: [companyId], references: [id])
  plateNumber        String
  status             String    @default("offline")
  vehicleType        String?
  driverId           String?   @unique
  deviceId           String?   @unique
```

```
  trackerToken       String?
  deviceLastSeenAt   DateTime?
  deviceBatteryLevel Float?
  createdAt          DateTime  @default(now())
  updatedAt          DateTime  @updatedAt
  pings              LocationPing[]
  trips              Trip[]
  @@unique([companyId, plateNumber])
  @@map("vehicles")
}

model LocationPing {
  id          String   @id @default(cuid())
  vehicleId   String
  vehicle     Vehicle  @relation(fields: [vehicleId], references: [id], onDelete: Cascade)
  latitude    Float
  longitude   Float
  speedKph    Float?
  heading     Float?
  accuracyM   Float?
  isMocked    Boolean  @default(false)
  source      String   @default("manual")
  recordedAt  DateTime @default(now())
  @@index([vehicleId, recordedAt(sort: Desc)])
  @@map("location_pings")
}
```

Then create the tables:

```
npx prisma migrate dev --name init      # creates the tables locally, writes prisma/migrations/
npx prisma studio                          # optional: browse the empty tables
```

**Check every field against the real model** in backend/models/ as you go (the table in section 3 lists them). A field missed here is silently dropped during the data copy.

# 5. Keep the API shape: the _id adapter

Mongoose documents serialise as `{ _id, ... }` and populated links as nested objects (e.g. `vehicleId: { _id, plateNumber }`). Prisma returns `{ id, ... }` and relations under their relation name. One helper keeps the responses identical:

```
// backend/lib/toApi.js — Prisma row -> the JSON shape the apps already expect
export function toApi(row) {
  if (Array.isArray(row)) return row.map(toApi);
  if (!row || typeof row !== "object" || row instanceof Date) return row;
  const out = {};
  for (const [k, v] of Object.entries(row)) {
    if (k === "id") out._id = v;
    else if (k === "passwordHash" || k === "data") continue;   // never leak these
    else out[k] = v && typeof v === "object" && !(v instanceof Date) ? toApi(v) : v;
  }
  return out;
}
```

Where Mongoose replaced a foreign key with the linked document (populate), select the relation and put it back under the old key, e.g. `{ ...trip, vehicleId: trip.vehicle }`. Also keep computed fields that are virtuals today: **avatarUrl** (User), **logoUrl** (Company), **receiptUrl**, **photoUrl**, which come from `uploadUrl(id)` in lib/uploadUrl.js.

Test the shape, not just the data: compare a few real responses before and after (section 10). The dashboard's TypeScript types in frontend/app/types/auth.ts list exactly what it expects.

# 6. Rewrite the queries (Mongoose to Prisma cheat sheet)

| Mongoose (today) | Prisma (after) |
|---|---|
| Vehicle.findById(id) | prisma.vehicle.findUnique({ where: { id } }) |
| Vehicle.findOne({ driverId }) | prisma.vehicle.findFirst({ where: { driverId } }) |
| Trip.find({ vehicleId: { $in: ids } }).sort({ startTime: -1 }).limit(100) | prisma.trip.findMany({ where: { vehicleId: { in: ids } }, orderBy: { startTime: 'desc' }, take: 100 }) |
| .populate('vehicleId', 'plateNumber') | include: { vehicle: { select: { id: true, plateNumber: true } } } |
| { endTime: { $exists: false } } | { endTime: null } |
| { $or: [a, b] } | { OR: [a, b] } |
| { recordedAt: { $gte: from, $lte: to } } | { recordedAt: { gte: from, lte: to } } |
| Model.create({...}) | prisma.model.create({ data: {...} }) |
| findOneAndUpdate(filter, {$set}, {new: true}) | prisma.model.update({ where, data }) (unique filter) or updateMany |
| doc.field = x; await doc.save() | prisma.model.update({ where: { id }, data: { field: x } }) |
| countDocuments(filter) | prisma.model.count({ where }) |
| Model.exists(filter) | (await prisma.model.findFirst({ where, select: { id: true } })) !== null |
| updateMany(filter, {$set}) | prisma.model.updateMany({ where, data }) |
| deleteOne / deleteMany | prisma.model.delete / deleteMany |
| aggregate([{ $match }, { $group: { _id: null, total: { $sum: '$x' } } }]) | prisma.model.aggregate({ where, _sum: { x: true } }) |
| aggregate with $group by field | prisma.model.groupBy({ by: ['field'], _sum: {...}, _count: true }) |
| Complex pipelines (dates, lookups) | prisma.$queryRaw`SELECT ... GROUP BY ...` (plain SQL, parameterised) |
| tenantScope(req) → { companyId } | Unchanged idea: spread into where: { ...tenantScope(req) } |

## The 10 aggregation pipelines to rewrite by hand

| File | What it computes | Suggested replacement |
|---|---|---|
| controllers/analyticsController.js (×2) | Vehicles per company; company growth by month | groupBy + raw SQL with date_trunc('month', created_at) |
| controllers/billingController.js | Total paid | aggregate _sum |
| controllers/companyController.js (×2) | Total trip distance; daily distance | aggregate _sum; raw SQL GROUP BY date for daily |
| controllers/expenseController.js | Total expenses | aggregate _sum |
| controllers/maintenanceController.js | Distance since odometer base | aggregate _sum |
| controllers/messageController.js | One row per driver thread, latest message + unread count | Raw SQL with DISTINCT ON (driver_id) and a COUNT FILTER |
| lib/driverScore.js (×2) | Events per driver; km per driver | groupBy by driverId (_count, _sum) |

# 7. Replace MongoDB-only features

| MongoDB feature | Where | PostgreSQL replacement |
|---|---|---|
| TTL index: delete resolved alerts after 30 min | models/Alert.js (resolvedAt) | Add to the existing node-cron jobs in Index.js, every 5 min: DELETE FROM alerts WHERE resolved_at < now() - interval '30 minutes' |
| Mongoose virtuals avatarUrl / logoUrl | models/User.js, Company.js | Compute in toApi / the controller with uploadUrl(row.avatarUploadId) |
| Schema defaults on nested objects | Company alertSettings etc. | lib/companySettings.js already fills defaults on read; keep it |
| Unique indexes | email, slug, trackingToken, reference… | @unique / @@unique in Prisma (duplicates fail the data copy: clean first) |
| save() with validation | everywhere | Validate in code before create/update (lengths, enums, ranges) |
| Automatic ObjectId on new docs | everywhere | @default(cuid()) for new rows (section 4); existing ids are copied as-is |

**Location pings:** the tracking endpoint writes several rows per vehicle per minute and reads the previous ping on every write. Keep the (vehicle_id, recorded_at DESC) index, and batch inserts with createMany when the phone uploads an offline backlog.

# 8. Order of work: file by file

Convert in this order so you can test each part end to end before moving on. Counts are Mongoose calls in each file today.

| Step | Files | Calls | Test after this step |
|---|---|---|---|
| 1 Auth & users | middleware/auth.js, controllers/authController.js, userController.js | 2 + 17 + 11 | Login, refresh, register company, add driver, profile |
| 2 Companies & settings | companyController.js, settingsCompanyController.js, settingsController.js, lib/companySettings.js | 19 + 8 + 2 + 2 | Company settings page, platform companies list |
| 3 Vehicles & uploads | vehicleController.js, uploadController.js | 20 + 5 | Vehicles page, photos, logos |
| 4 Tracking (critical) | trackingController.js, lib/alerts.js | 33 + 4 | Driver app sharing, Live Map, trips open/close, speeding |
| 5 Trips & reports | tripController.js, reportController.js, lib/driverScore.js | 17 + 8 + 3 | Trips page, Summary report, Excel export, driver scores |
| 6 Alerts & messages | alertController.js, messageController.js | 9 + 9 | Alerts page, driver delete, chat both sides |
| 7 Jobs & routes | jobController.js, routeController.js, geofenceController.js | 26 + 7 + 5 | Dispatch, proof of delivery, customer link |
| 8 Expenses, inspections, maintenance | expenseController.js, inspectionController.js, maintenanceController.js | 8 + 7 + 12 | Each page + driver app screens |
| 9 Billing & analytics | billingController.js, analyticsController.js | 13 + 9 | Paystack checkout/webhook, platform analytics |
| 10 AI agent | agentController.js, services/aiAnomalyDetector.js, services/fleetAssistant.js | 4 + 7 + 6 | Agent scan, assistant answers |

Finally remove Mongoose: delete **backend/models/** and **config/db.js**, run `npm uninstall mongoose`, and drop `MONGODB_URI` once the backup period is over.

# 9. Copy the data across

Write **backend/scripts/mongoToPostgres.js**. It reads every collection with the official MongoDB driver and inserts into PostgreSQL with Prisma, **parents before children** so foreign keys resolve. Run it against an empty PostgreSQL database.

```
// backend/scripts/mongoToPostgres.js  (outline)
import "dotenv/config";
import { MongoClient } from "mongodb";
import { prisma } from "../lib/prisma.js";

const mongo = await new MongoClient(process.env.MONGODB_URI).connect();
const db = mongo.db();
const id = (v) => (v ? String(v) : null);       // ObjectId -> text

// Parents first: uploads & companies & users, then everything that links to them.
const ORDER = ["companies", "uploads", "users", "geofences", "vehicles", "routes", "trips",
  "locationpings", "drivingevents", "alerts", "jobs", "inspections", "routecompletions",
  "expenses", "maintenancerecords", "messages", "refreshtokens", "transactions",
  "platformsettings", "agentscanlogs"];

const MAP = {
  vehicles: (d) => ({
    id: id(d._id), companyId: id(d.companyId), plateNumber: d.plateNumber, status: d.status,
    vehicleType: d.vehicleType ?? null, driverId: id(d.driverId), deviceId: d.deviceId ?? null,
    trackerToken: d.trackerToken ?? null, deviceLastSeenAt: d.device?.lastSeenAt ?? null,
    createdAt: d.createdAt, updatedAt: d.updatedAt ?? d.createdAt,
  }),
  // ... one mapper per collection, field for field
};

for (const name of ORDER) {
  const docs = await db.collection(name).find().toArray();
  const rows = docs.map(MAP[name]);
  for (let i = 0; i < rows.length; i += 1000) {
    await prisma[MODEL_OF[name]].createMany({ data: rows.slice(i, i + 1000) });
  }
  console.log(name, docs.length, "->", await prisma[MODEL_OF[name]].count());
}
```

## Watch out for

- **Circular links:** Company.logoUploadId → Upload and Upload.companyId → Company. Insert companies without logoUploadId, insert uploads, then update the logos. Same for Company.dailyStartPoint → Geofence.
- **Orphans:** MongoDB doesn't enforce links, PostgreSQL does. A trip pointing at a deleted vehicle will fail. Count orphans first (a $lookup or a quick script) and decide: delete, or set the link to null.
- **Duplicates on unique fields** (email, slug, plate per company, trackingToken). Clean them before copying.
- **Uploads:** read `data` as a Buffer (it's excluded by default in Mongoose; the raw driver returns it).
- **Dates:** missing updatedAt on old documents → fall back to createdAt.

## Verify

Every collection must have the same number of rows on both sides. Today's counts, for reference:

| Collection | Documents | Collection | Documents |
|---|---|---|---|
| locationpings | 16,783 | messages | 8 |

| Collection | Documents | Collection | Documents |
|---|---|---|---|
| agentscanlogs | 543 | vehicles | 7 |
| refreshtokens | 233 | uploads | 6 |
| trips | 98 | expenses | 5 |
| drivingevents | 85 | companies | 4 |
| alerts | 79 | transactions | 4 |
| users | 14 | routes / geofences | 2 / 2 |
| jobs / inspections | 1 / 1 | routecompletions / platformsettings | 1 / 1 |

Then spot-check: pick 3 random records per table and compare every field.

# 10. Test everything

## Compare responses before and after

With MongoDB and PostgreSQL holding the same data, call the same endpoints on both versions and diff the JSON. A tiny script that logs in, fetches a list of endpoints and writes each response to a file makes this repeatable:

```
GET /api/companies/me        GET /api/vehicles            GET /api/tracking/live
GET /api/trips?from=..&to=.. GET /api/alerts              GET /api/jobs
GET /api/reports/summary     GET /api/messages/threads    GET /api/users/drivers
GET /api/expenses            GET /api/inspections         GET /api/analytics (platform)
```

## End-to-end flows to click through

- Register a company, log in as owner, admin, dispatcher, viewer and platform admin.
- Driver app: log in, share location, drive (or replay pings), see the trip open and close; driving mode; end a trip.
- Live Map: vehicles, address lookup, create a geofence / route from a map click.
- Alerts: speeding/geofence alerts appear, resolve one, check it disappears after 30 min (the new cron job).
- Jobs: create, optimise, complete with photo + signature, customer tracking link.
- Uploads: avatar, company logo, receipt and inspection photos display.
- Reports: Summary page, Excel export, PDF print, driver scores.
- Billing: a Paystack test checkout and webhook.

# 11. Switch over on Render (and roll back if needed)

1. **Announce a short maintenance window** (15 minutes is plenty for 4 MB). During it, drivers' phones keep buffering locations offline (up to 6 hours), so no tracking is lost.
2. **Final backup:** mongodump again, so the copy uses the very latest data.
3. **Run the copy** into the production PostgreSQL: `node scripts/mongoToPostgres.js`, then check the counts.
4. **On Render:** add `DATABASE_URL`, set the build command to `npm install && npx prisma generate && npx prisma migrate deploy`, and deploy the merged main.
5. **Smoke test** production: login, Live Map, one driver sharing location, alerts, a report.

6 **Keep MongoDB** untouched for 2 to 4 weeks. Only then remove MONGODB_URI from Render and pause the Atlas cluster.

### Rollback plan

If something serious breaks after the switch: redeploy the last MongoDB-based commit on Render (one click, "Rollback" in the deploy history). Data written to PostgreSQL in between would need copying back; that's why the switch-over should happen in a quiet hour and be verified straight away.

## 12. Checklist

| Done | Step |
|---|---|
| [ ] | mongodump backup taken and stored safely |
| [ ] | Branch feature/postgres-migration created from up-to-date main |
| [ ] | PostgreSQL database created (Neon recommended), DATABASE_URL in .env |
| [ ] | Prisma installed; schema.prisma covers all 20 models; migrate dev runs clean |
| [ ] | lib/prisma.js and lib/toApi.js in place |
| [ ] | Steps 1 to 10 of section 8 converted and tested one by one |
| [ ] | 10 aggregation pipelines rewritten (section 6) |
| [ ] | Alert 30-minute cleanup moved to a cron job |
| [ ] | Orphans and duplicates cleaned in MongoDB |
| [ ] | mongoToPostgres.js copies everything; counts match |
| [ ] | API responses identical before/after for the key endpoints |
| [ ] | End-to-end flows clicked through (section 10) |
| [ ] | PR into dev-francis → dev-staging → main (Mike approves) |
| [ ] | Render switched over, smoke-tested, MongoDB kept as backup for 2 to 4 weeks |
| [ ] | Mongoose and backend/models removed afterwards |